

CAPÍTULO 5

ESTRUCTURAS INDEXADAS, LINEALES Y GESTIÓN DE MEMORIA



5.1. Lección 12

En esta lección aprenderemos a usar *array*, un tipo de colección nativa que proveen los lenguajes de programación. Además, desarrollaremos una API y dos TAD que se convertirán en herramientas esenciales a la hora de resolver problemas.

5.1.1. Array

Un *array* representa un conjunto finito de variables del mismo tipo de dato, ubicadas una a continuación de otra en la memoria principal de la computadora.

El hecho de que todas las variables del conjunto ocupen posiciones consecutivas de memoria es fundamental, pues nos permite acceder a cualquiera de éstas indicando el nombre del *array*, más un índice haciendo referencia a la variable del conjunto que queremos utilizar.

En las siguientes líneas de código, declaramos un *array* tipo `int` cuya capacidad le permite contener tres valores enteros.



Lección 12

```
int a[3];
a[0]=123;
a[1]=456;
a[2]=789;
```

Gráficamente podemos representar el *array* *a* vertical u horizontalmente, tal como se ilustra en la siguiente figura.

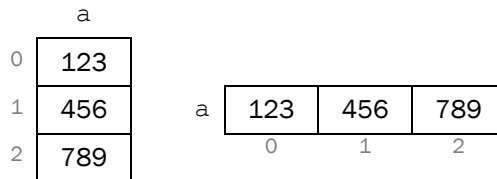


Figura 5.1. Diferentes modos de representación de un *array*

A continuación, mostramos por pantalla los valores que contiene el *array* *a*.

```
cout << a[0] << endl;
cout << a[1] << endl;
cout << a[2] << endl;
```

```
for(int i=0; i<3; i++)
{
    cout << a[i] << endl;
}
```

En el código de la izquierda, accedemos manualmente a cada una de las posiciones del *array*. A la derecha, lo hacemos programáticamente.

Como un *array* representa un conjunto de datos diremos que es una *estructura*. Y la posibilidad utilizar un índice para acceder a cada uno de los elementos del conjunto lo convierte en una *estructura indexada*. Finalmente, como se trata de un conjunto de datos, también es una colección.

5.1.1.1. Capacidad del *array*

La *capacidad* se define en el momento de la declaración del *array*, y establece cuál será la cantidad de variables (elementos) que el *array* podrá contener. Este valor no puede ser alterado, razón por la cual los *array* son *estructuras estáticas*.

5.1.1.2. Inicializar un array a partir un conjunto de valores

En el ejemplo anterior, declaramos el *array* *a* como `int[3]` y le asignamos valor a cada uno de sus elementos, que se identifican como `a[0]`, `a[1]` y `a[2]`.

Podríamos simplificar estas líneas y asignar, directamente sobre el *array*, el conjunto completo de valores que queremos dar a sus elementos:

```
int a[3] = {123,456,789};
```

Incluso, podríamos prescindir de indicar la longitud del *array*, ya que el lenguaje puede inferirla a partir de la cantidad de elementos del conjunto.

```
int a[] = {123,456,789};
```

En ambos casos, `a[0]`, `a[1]` y `a[2]` tomarán, respectivamente, los valores 123, 456 y 789.

5.1.1.3. Inicialización programática

Si la capacidad del *array* es demasiado grande podría resultar engorroso asignarle manualmente el conjunto de valores iniciales. Tal vez, el valor que necesitemos asignar a cada elemento esté en función de la posición que dicho elemento ocupa dentro del conjunto. Si este fuera el caso, tendríamos que implementar una inicialización programática, como vemos a continuación.

```
int a[100];
for(int i=0;i<100;i++)
{
    a[i]=0; // asigno 0
}
```

```
int a[100];
for(int i=0;i<100;i++)
{
    a[i]=i; // asigno i
}
```

En el ejemplo de la izquierda, declaramos un *array* con capacidad para contener 100 valores tipo `int` e inicializamos cada uno de estos valores con 0 (cero).

En el ejemplo de la derecha, declaramos un *array* con las mismas características, e inicializamos cada uno de sus elementos con el valor de la posición que ocupa dentro del conjunto.

5.1.1.4. Longitud del array

Al declarar un *array* establecemos su capacidad. Por ejemplo, cuando escribimos `int a[10]` declaramos el *array* `a` cuya capacidad física que le permitirá contener 10 elementos tipo `int`. Este *array* ocupará 40 bytes de memoria, calculados como: `10*sizeof(int)`.

La capacidad es una característica física del *array*, y una vez establecida permanecerá inalterable. En cambio, la *longitud* es un parámetro que nosotros mismos debemos administrar para conocer, en un determinado momento, cuántos elementos del *array* efectivamente están siendo utilizados; más allá de cuál sea su capacidad.

Al declarar un *array*, inicialmente vacío, consideraremos que su longitud es 0. Luego, cada vez que le agreguemos un elemento, incrementaremos su longitud para reflejar que el *array* tiene un elemento más. La longitud máxima estará limitada por su capacidad, pues no podemos agregar más elementos que los que, físicamente, el *array* puede contener.

Podemos agregar o quitar elementos, aumentando o disminuyendo su longitud. Pero en todo momento el *array* ocupará la misma cantidad de memoria, que calcularemos como: $n \times \text{sizeof}(T)$ bytes, donde n es su capacidad y T , el tipo de dato de sus elementos. Recordemos que todos los elementos del *array* son del mismo tipo.

En otras palabras, si declaramos un *array* de 100 elementos y sólo utilizamos los primeros 5, su longitud será 5. Si agregamos 10 elementos más, su longitud será 15. Y si agregamos otros 5, su longitud será 20. Luego, si removemos 12 elementos, su longitud pasará a ser 8.

5.1.1.5. Arrays de estructuras

Los *array* pueden ser de cualquier tipo de dato, incluyendo estructuras.

Veamos la estructura `Persona`, con su correspondiente función de inicialización, la cual utilizaremos para crear un *array* de personas.

```

struct Persona
{
    string nombre;
    int dni;
    string direccion;
    int fechaNac;
};

Persona persona(string n,int dni,string dir,int fec)
{
    return {n,dni,dir,fec};
}

```

Podemos crear e inicializar un *array* de personas así:

```

Persona arr[10]; // un array de 10 personas
arr[0] = persona("Juan",12232554,"Libertador 6",19831223);
arr[1] = persona("Pedro",23443233,"Areco 4442",19701005);
arr[2] = persona("Pablo",21554321,"Los Sauces 12",19990128);
// :           :           :           :
arr[9] = persona("Carlos",23312554,"El Tala 23",19950304);

```

Otra forma de hacer lo mismo sería declarando el conjunto de personas que queremos contener dentro del *array*.

```

Persona arr[] = {
    persona("Juan",12232554,"Libertador 6",19831223)
    ,persona("Pedro",23443233,"Areco 4442",19701005)
    ,persona("Pablo",21554321,"Los Sauces 12",19990128)
    // :
    ,persona("Carlos",23312554,"El Tala 23",19950304)
};

```

Gráficamente, representaremos el *array* `arr` tal como vemos a continuación.

	nombre	dni	direccion	fecNac
0	Juan	12232554	Libertador 6	19831223
1	Pedro	23443233	Areco 4442	19701005
2	Pablo	21554321	Los Sauces 12	19990128
:	:	:	:	:
9	Carlos	23312554	El Tala 23	19950304

Figura 5.2. Representación de un array de estructuras

Para acceder a cualquier campo de un elemento del *array* indicaremos el nombre del *array* (*arr*), el índice que direcciona cuál es el elemento, y el nombre del campo cuyo valor queremos consultar o modificar.

```
arr[1].direccion = "Av. San Martin 245, 5to. B";
```

Luego de esta sentencia, habremos cambiado el valor del campo *direccion* de la persona representada por el segundo elemento del *array* (Pedro).

5.1.1.6. Arrays multidimensionales

Cuando el tipo de dato de un *array* es, en sí mismo, otro *array*, se trata de un *array multidimensional*. Así, `int a[3][3]` declara un *array* de *array* de `int`. Gráficamente podemos imaginarlo como se muestra en la figura de la izquierda, y más simplificada, como se observa en la figura del centro.

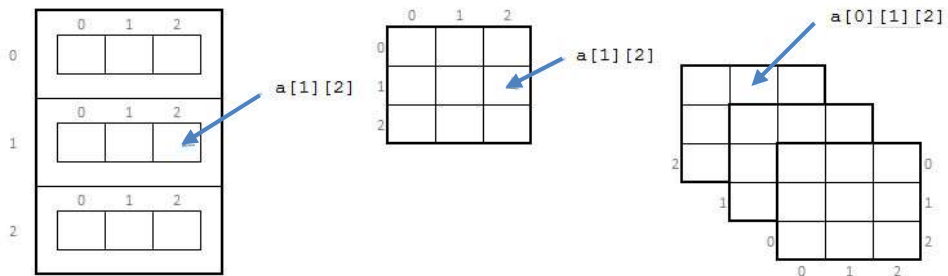


Figura 5.3. Arrays multidimensionales

Cabe aclarar que no tiene por qué coincidir la cantidad de filas y columnas. Podríamos haber declarado: `int a[3][8]` y tendríamos un *array* multidimensional de 3 filas y 8 columnas.

Cuando el *array* tiene dos dimensiones, como es el caso del *array* `a`, lo llamamos *matriz*. Para acceder a un elemento de la matriz, debemos indicar fila y columna de la celda que contiene el elemento cuyo valor queremos consultar o modificar.

En cambio, si declaramos `int a[3][3][3]` tendremos un *array* de *array* de *array* de `int`. Ahora, el *array* `a` tendrá tres dimensiones. Gráficamente, lo podemos imaginar como un *cubo*, tal como se ilustra en la imagen de la derecha de la figura 5.3. Para acceder a un elemento del cubo debemos indicar: fila, columna y profundidad.

5.1.2. Operaciones sobre arrays

Para utilizar *arrays* necesitaremos disponer de un conjunto de operaciones básicas que implementaremos como funciones; las cuales, más adelante, formarán parte de la API de tratamiento de *arrays* que desarrollaremos en este mismo capítulo.

Agregar un elemento, determinar si el *array* contiene o no un elemento especificado e insertar o eliminar un elemento en una determinada posición son algunas de las operaciones básicas que debemos resolver para agilizar y facilitar la operatoria con *arrays* en nuestros algoritmos.

Para estudiar el tema, trabajaremos con *arrays* tipo `int`. Luego, plantearemos una API genérica basada en *templates*. Todas estas funciones deben administrar la longitud del *array*. Utilizaremos la variable `len` (abreviatura de *length*) para hacer referencia a dicha longitud.

5.1.2.1. Agregar un elemento al final de un array

La función `agregar` permite agregar un elemento al final de un *array*, incrementa su longitud y retorna la posición en la cual quedó asignado el elemento. Aunque esta función modifica el *array*, no necesitaremos agregarle & al parámetro `a`, porque como veremos más adelante, el *array* es, en sí mismo, una referencia.

```
int agregar(int a[],int& len,int v)
{
    a[len] = v;
    len++;
    return len-1;
}
```

Al disponer de esta función, podríamos escribir el siguiente programa:

```
int main()
{
    int arr[100];
    int len=0;

    agregar(arr, len, 2);
    agregar(arr, len, 3);
    agregar(arr, len, 5);
    agregar(arr, len, 7);
    agregar(arr, len, 11);

    for(int i=0; i<len; i++)
    {
        cout << arr[i] << endl;
    }

    return 0;
}
```

5.1.2.2. Determinar si el array contiene un elemento especificado

La función `buscar` recorre un *array* para determinar si contiene o no un valor especificado, y retorna la posición donde lo encontró, o un valor negativo para indicar que el *array* no contiene dicho valor.

```
int buscar(int a[], int len, int v)
{
    int i=0;
    while(i<len && a[i]!=v)
    {
        i++;
    }

    return i<len?i:-1;
}
```

Un ejemplo de uso de esta función podría ser:


```

int main()
{
    int arr[100];
    int len=0;

    agregar(arr,len,2);
    agregar(arr,len,3);
    agregar(arr,len,5);
    agregar(arr,len,7);
    agregar(arr,len,11);

    // el usuario ingresa un valor
    int v;
    cout << "Ingrese un valor: " << endl;
    cin >> v;

    // el array contiene el valor que ingreso el usuario?
    int pos = buscar(arr,len,v);
    if( pos>=0 )
    {
        cout<<v<<" se encontro en la posicion:"<<pos<<endl;
    }
    else
    {
        cout << v << " no se encontro en el array" << endl;
    }

    return 0;
}

```

5.1.2.3. Insertar un elemento en una determinada posición

La función `insertar` inserta el elemento `v` en la posición `p`. Para esto, previamente debe desplazar hacia el final del `array` a todos los elementos ubicados entre las posiciones `p` y `len-1`, incrementando así su longitud.

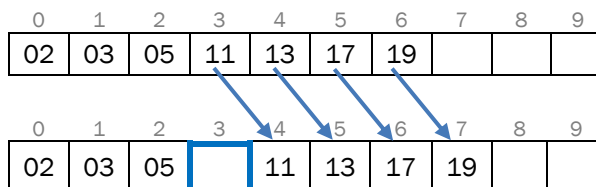


Figura 5.4. La función `insertar` desplaza los elementos del `array` una posición hacia el final

Suponiendo que `p` es 3, la imagen muestra cómo `insertar` desplazó una posición hacia el final a todos los elementos del `array` ubicados entre las posiciones `p` y `len-1`; liberando, de este modo, la posición `p` para colocar allí al nuevo elemento `v`.

```
void insertar(int a[],int& len,int v,int p)
{
    for(int i=len; i>p; i--)
    {
        a[i]=a[i-1];
    }

    a[p]=v;
    len++;
}
```

El siguiente programa utiliza `insertar` para invertir una lista de números. La Salida será: 5, 4, 3, 2, 1.

```
int arr[100];
int len=0;

insertar(arr,len,1,0);
insertar(arr,len,2,0);
insertar(arr,len,3,0);
insertar(arr,len,4,0);
insertar(arr,len,5,0);

for(int i=0; i<len; i++)
{
    cout << arr[i] << endl;
}
```

5.1.2.4. Eliminar el elemento ubicado en una determinada posición

La función `eliminar` es la inversa de `insertar`. Desplaza una posición hacia el inicio los elementos del `array` ubicados entre las posiciones `p+1` y `len-1`, retornando el valor que se ubicaba en la posición `p` y disminuyendo la longitud del `array`.

Si `p` fuera 3, vemos cómo el valor 13 reemplaza al elemento 11, que ya no forma parte del `array`. La longitud del `array` pasó de 7 a 6.

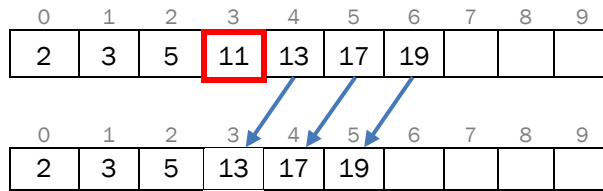


Figura 5.5. La función eliminar desplaza los elementos del array una posición hacia el inicio

```
int eliminar(int a[],int& len,int p)
{
    int ret = a[p];
    for(int i=p+1; i<len; i++)
    {
        a[i-1]=a[i];
    }

    len--;
    return ret;
}
```

El siguiente programa muestra y elimina cada uno de los elementos de un array. ¿Cuál será la salida por pantalla?

```
int arr[100];
int len=0;

// metemos datos en el array
insertar(arr,len,1,0);
insertar(arr,len,2,0);
insertar(arr,len,3,0);
insertar(arr,len,4,0);
insertar(arr,len,5,0);

while(len>0)
{
    cout << eliminar(arr,len,0) << endl;
}
```

5.1.3. Actividad práctica: API de operaciones sobre arrays



Las siguientes funciones son la generalización de las operaciones que estudiamos previamente. Su implementación quedará a cargo del lector, aceptando que las mismas formarán parte de nuestra biblioteca de funciones.

A continuación, y a modo de ejemplo, veremos cómo desarrollar una de las funciones especificadas en la API.

La función `add`, que agrega un elemento al final de un `array` y retorna la posición en quedó colocado, tiene el siguiente prototipo:

```
int add(T arr[],int len,T e);
```

Un ejemplo de uso de esta función sería el siguiente:

```
// array y longitud
string a[10];
int len=0;

// agrego elementos
add<string>(a,len,"John");
add<string>(a,len,"Paul");
add<string>(a,len,"George");
add<string>(a,len,"Ringo");

// recorro y muestro
for(int i=0;i<len;i++)
{
    cout << a[i] << endl;
}
```



API de operaciones
sobre arrays

La codificaremos de la siguiente manera:

```
template<typename T>
int add(T arr[],int len,T e)
{
    arr[len++] = e;
    return len;
}
```

5.1.4. Autoevaluación y ejercicios

*Autoevaluación**Ejercicios*

5.2. Lección 13

Aquí, veremos cómo acceder a los espacios de memoria que representan las variables, pero lo haremos a través de sus direcciones. De este modo, evitaremos trabajar con copias de sus valores ya que trataremos directamente con los datos. Así lograremos, por ejemplo, que cualquier modificación que realicemos sobre los elementos de una colección quede reflejada automáticamente, sin necesidad de volver a asignar el elemento modificado.

5.2.1. Gestión de memoria

En varias oportunidades mencionamos que las variables representan un espacio de la memoria principal de la computadora, y su tipo de dato establece cuántos bytes serán destinados a la variable para que ésta pueda contener un valor.

En el siguiente bloque de código, declaramos las variables `i`, `s` y `c`, cuyos tipos de dato son `int`, `short` y `char` respectivamente, y les asignamos valor.

*Lección 13*

```
int i = 54321;
short s = 25;
char c = 'A';
```

Podemos imaginar que la memoria de la computadora es una grilla compuesta por celdas, donde cada una representa 1 byte y se identifica por un número que llamaremos *dirección*: su *dirección de memoria*.

Las variables *i*, *s* y *c*, según sus tipos de dato, representan respectivamente 4, 2 y 1 bytes de memoria. No podemos saber qué celdas les serán asignadas, pero sí estamos en condiciones de asegurar que todos sus bytes serán contiguos.

0	1	2	3	4	5	6	7	8	9
	25								
10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29
		'A'							
30	31	32	33	34	35	36	37	38	39
				54321					

Figura 5.6. Mapa de memoria

En esta representación, suponemos y aceptamos que la dirección de memoria de la variable *s* es 1 y, por tratarse de un `short`, ocupa los bytes 1 y 2. La dirección de memoria de *c* es 22, la dirección de *i* es 34, y ocupa los bytes 34, 35, 36 y 37.

5.2.1.1. Punteros

Así como en una variable podemos guardar un valor, también podemos guardar una dirección de memoria. Las variables preparadas para guardar direcciones de memoria se llaman *punteros*.

Para declarar un puntero (una variable tipo puntero) debemos agregar `*` (asterisco) al tipo de dato de la variable. Por ejemplo, la variable *p*, declarada como vemos a continuación, podrá guardar una dirección de memoria a partir de la cual se aloja un valor tipo `int`. A fin de proporcionar un valor inicial a los punteros, podemos utilizar la constante `NULL` que representa una dirección nula de memoria.

```
int* p = NULL;
```

Siguiendo con el ejemplo, observamos que a partir de la dirección de memoria 34 se aloja el valor 54321, cuyo tipo de dato es `int`. La variable `p`, así como la declaramos, podría guardar la dirección 34. Pero surgen las siguientes preguntas: ¿cómo obtenemos dicha dirección? y ¿para qué? El cómo lo estudiaremos a continuación. El *para qué* lo iremos descubriendo más adelante.

5.2.1.2. Operador de dirección (&)

Al anteponer el operador `&` (*ampersand*) a una variable obtenemos su dirección de memoria, que podemos guardar en un puntero. Lo llamaremos *operador de dirección*.

Continuando con el ejemplo, podríamos asignar:

```
int* p = &i; // asignamos a p la direccion de i: 34
```

5.2.1.3. Operador de indirección o contenido (*)

Al anteponer `*` (asterisco) a una variable tipo puntero obtendremos el valor que se aloja a partir de la dirección de memoria señalada por dicho puntero. Previamente, asignamos a `p` la dirección de `i`, a partir de la cual se aloja el valor 54321. Entonces, `*p` es 54321. Diremos que `*` (asterisco) es el *operador de contenido*.

```
cout << *p << endl; // muestra: 54321
```

Si `p` es la dirección de `i`, podemos asignar un nuevo valor a `i` a través de su dirección de memoria, así:

```
*p = 112233; // asignamos un nuevo valor
```

En la jerga, hablamos de “el contenido de”. Si `p` es un puntero, entonces, decimos que `*p` “es el contenido de `p`”. Pero esto puede resultar algo confuso, ya que al ser `p` un puntero su contenido es una dirección de memoria. La expresión “el contenido de `p`” debe interpretarse como *el contenido que se aloja en la memoria que está siendo direccionada por `p`*.

5.2.1.4. Funciones que reciben punteros como parámetros

Al tener la posibilidad de manipular espacios de memoria a través de sus direcciones, las funciones que reciben punteros como parámetros podrán modificar los valores que dichos punteros direccionan tal como si los recibieran por referencia.

A continuación, veremos dos versiones de la función `permutar` que ya hemos estudiado en el capítulo 1, que recibe dos parámetros y permuta sus valores.

La versión de la izquierda ya la analizamos: recibe dos parámetros por referencia y simplemente los permuta. En cambio, la versión de la derecha recibe las direcciones de memoria de dichos valores, y permuta sus contenidos.

```
void permutar1(int& a,int& b)
{
    int aux=a;
    a=b;
    b=aux;
}
```

```
void permutar2(int* a,int* b)
{
    int aux=*a;
    *a=*b;
    *b=aux;
}
```

El código de `permutar2` debe leerse así: asigno a `aux` el contenido de `a`; al contenido de `a` le asigno el contenido de `b`; al contenido de `b` le asigno el valor de `aux`.

Aunque ambas versiones hacen exactamente lo mismo, como sus prototipos son diferentes, el modo de invocarlas también lo será.

```
int main()
{
    int x=5,y=10;

    // invoco a permutar1
    permutar1(x,y);

    // salida: 10,5
    cout << x << "," << y;
}
```

```
int main()
{
    int x=5,y=10;

    // invoco a permutar2
    permutar2(&x,&y);

    // salida: 10,5
    cout << x << "," << y;
}
```

Al invocar a `permutar2` (a la derecha) le pasamos como argumento las direcciones de memoria de `x` e `y`, que obtenemos al anteponer el operador `&`.

5.2.1.5. Punteros a punteros

Sabemos que un puntero puede contener la dirección de memoria de una variable. Ahora bien, ¿puede un puntero contener la dirección de otro puntero?

En el siguiente código declaramos las variables *i*, *p* y *q*. Asignamos a *p* la dirección de *i* y a *q* la dirección de *p*. El tipo de dato de *q* será: `int**`, pues tendrá la dirección de un puntero a entero. Diremos que *q* es un *puntero a puntero a int*.

```
int i = 10;
int* p = &i;
int** q = &p;
```

Gráficamente vemos que *p* guarda la dirección de *i*, y *q* guarda la dirección de *p*.

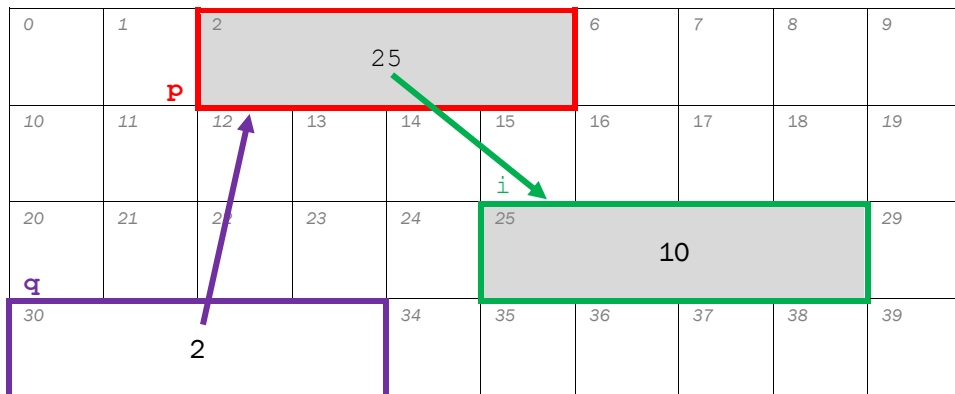


Figura 5.7. Mapa de memoria

Para acceder a *i* a través de *p* utilizamos el operador de dirección `*p`. Si quisiéramos acceder a *i* a través de *q* debemos hacerlo así: `**q`.

5.2.1.6. Punteros a estructuras y operador -> (flecha)

Los punteros permiten direccionar datos de cualquier tipo. En el siguiente código, declaramos la estructura `Persona`, una variable y un puntero.

```
struct Persona
{
```

```

    int dni;
    string nombre;
};

Persona per = {123, "Pablo"};
Persona* p = &per;

```

Existen dos modos de acceder a los campos de `per` a través del puntero `p`.

```
cout << (*p).nombre << endl;
```

```
cout << p->nombre << endl;
```

A la izquierda, accedemos al campo `nombre` aplicando el operador de contenido sobre `p`; y a la derecha lo hacemos mediante el operador *flecha*, que al no requerir paréntesis resulta más práctico y menos engorroso. También podemos asignar valor a cualquier campo de una estructura a través de su dirección de memoria.

```
(*p).nom = "Pedro";
```

```
p->nom = "Juan";
```

5.2.1.7. Punteros y arrays

Como mencionamos al inicio, un *array* representa un conjunto de variables del mismo tipo ubicadas en posiciones contiguas de memoria. Agregaremos también que la variable del *array* es por sí misma una referencia a la dirección de memoria del primero de sus elementos. Dicho de otro modo, si declaramos el *array* `short a[3]`, `a` es la dirección de `a[0]`. Incluiremos la declaración de este *array* al ejemplo con el que venimos trabajando.

```

int i = 54321;
short s = 25;
char c = 'A';

// nuevo
short a[3] = {123, 456, 789};

```

Ahora, aparecerá en nuestro mapa de memoria la variable `a` que representa un `array` de 3 `short`. Observemos que todos sus bytes son consecutivos.

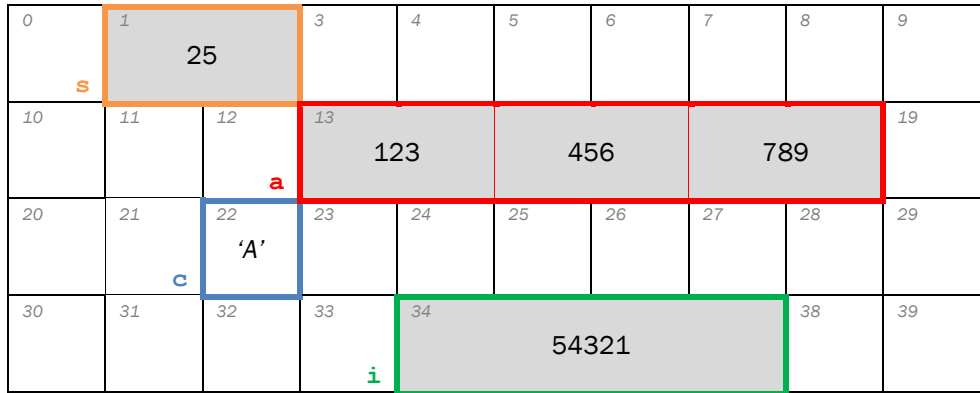


Figura 5.8. Mapa de memoria, incluye un `array`

Como `a` (el `array`) es la dirección del primero de sus elementos, resulta que `*a` es, según el ejemplo, 123; es decir: el contenido alojado en dicha dirección.

En otras palabras, `*a` y `a[0]` son sinónimos.

```
cout << *a << endl;    // salida: 123
cout << a[0] << endl;  // salida: 123
```

5.2.1.8. Aritmética de direcciones

Si `a` es la dirección del primer elemento del `array`, la dirección del segundo elemento será `a+1`. La dirección del tercer elemento será `a+2` y, genéricamente hablando, la dirección del i -ésimo elemento del `array` será `a+i`.

Usando el `array` `a` declarado antes, ambos códigos arrojarán la misma salida.

```
for(int i=0; i<3; i++)
{
    cout << a[i] << endl;
}
```

```
for(int i=0; i<3; i++)
{
    cout << *(a+i) << endl;
}
```

Observemos que `*(a+i)` no es lo mismo que `*a+i`. En el primer caso, hablamos del *i*-ésimo elemento del *array* *a*. En el segundo, hablamos del primer elemento del *array* incrementado en una unidad.

Continuando con el ejemplo, el código de la izquierda mostrará 123, 456, 789, mientras que el código de la derecha arrojará 123, 124, 125.

```
for(int i=0; i<3; i++)
{
    cout << *(a+i) << endl;
}
```

```
for(int i=0; i<3; i++)
{
    cout << *a+i << endl;
}
```

5.2.1.9. Memoria estática

Todas las variables que utilizamos, independientemente de cuál sea su tipo de dato, representan una porción de memoria estática. El simple hecho de declarar una variable implica reservar un espacio de memoria durante toda la ejecución del bloque de código dentro del cual, la variable, fue declarada.

Cuando declaramos `int i` reservamos 4 bytes para guardar un valor tipo entero. Si declaramos `short a[3]` reservamos 6 bytes para guardar 3 `short`. Al declarar `int* p` reservamos 4 bytes para guardar una dirección de memoria. Los punteros siempre ocuparán 4 bytes, independientemente de cuál sea su tipo de dato.

5.2.1.10. Memoria dinámica

La memoria puede ser asignada dinámicamente siempre que el programa así lo requiera; y cuando deje de tener utilidad ser devuelta al sistema operativo para que la destine a otros procesos. Para esto existen dos operadores: `new` y `delete`, que permiten reservar y liberar (pedir y devolver) memoria, respectivamente.

En el siguiente ejemplo, pedimos un espacio de memoria para almacenar un `int`, utilizamos dicho espacio; y finalmente, lo liberamos.

```
// declaro una variable tipo puntero
int* p = NULL;

// reservo (solicito) memoria dinamicamente
p = new int();
```

```
// uso el espacio de memoria solicitado
*p = 10;
cout << *p << endl; // SALIDA: 10

// libero (devuelvo) la memoria
delete p;
```

Para comprender este proceso, debemos analizar cómo quedará el mapa de memoria luego de ejecutar cada una estas sentencias.

Comenzamos declarando el puntero `p` al cual le asignamos el valor `NULL`.

```
// declaro una variable tipo puntero
int* p = NULL;
```

0	1	2	NULL			6	7	8	9
	P								
10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29
30	31	32	33	34	35	36	37	38	39

Figura 5.9. Mapa de memoria, ilustra un puntero

Luego, usamos el operador `new` para solicitar memoria dinámicamente.

El operador `new` retorna la dirección de memoria del espacio obtenido; esta dirección la asignamos a la variable `p`.

```
// reservo memoria dinamicamente
p = new int();
```

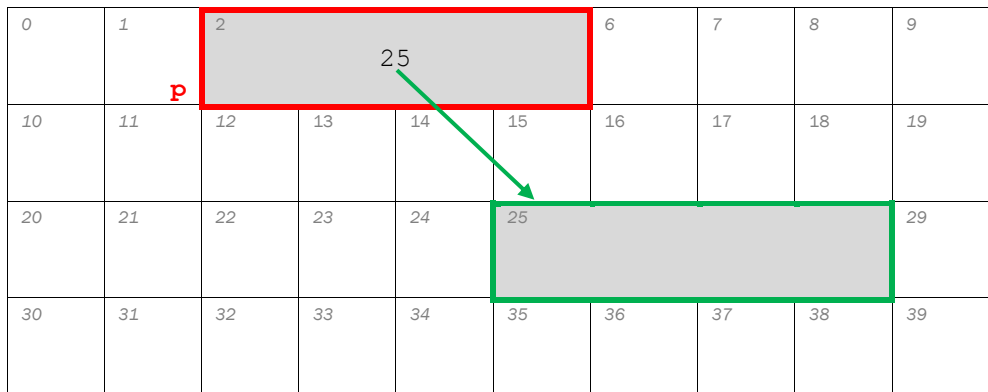


Figura 5.10. Mapa de memoria, ilustra memoria dinámica

A continuación, asignamos el valor 10 al espacio de memoria direccionado por `p`, que según el ejemplo es 25. Dicho espacio fue gestionado dinámicamente.

```
// uso el espacio de memoria
*p = 10;
cout << *p << endl; // SALIDA: 10
```

Observemos la diferencia que existe entre el espacio de memoria direccionado por `p` y la memoria representada por la variable `i` en la figura 5.7. En este caso, al no disponer de una variable que lo represente, el único modo de utilizar dicho espacio de memoria será a través de su dirección.

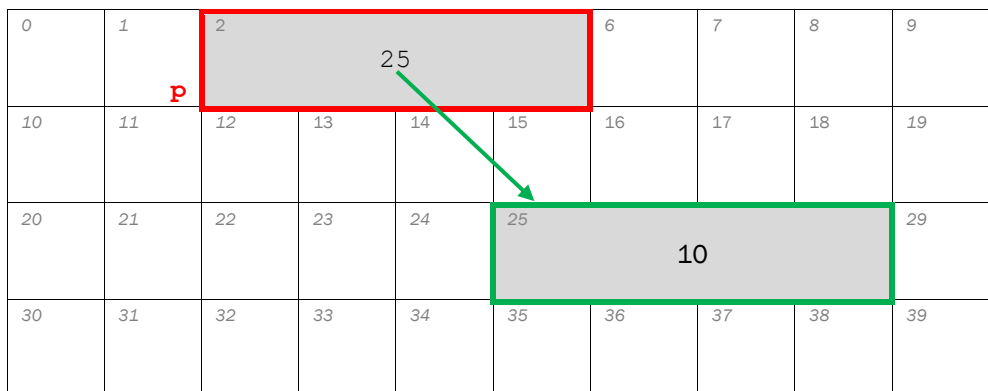


Figura 5.11. Mapa de memoria, manipula memoria dinámica

Por último, usamos `delete` para liberar la memoria que obtuvimos con `new`.

```
// libero la memoria
delete p;
```

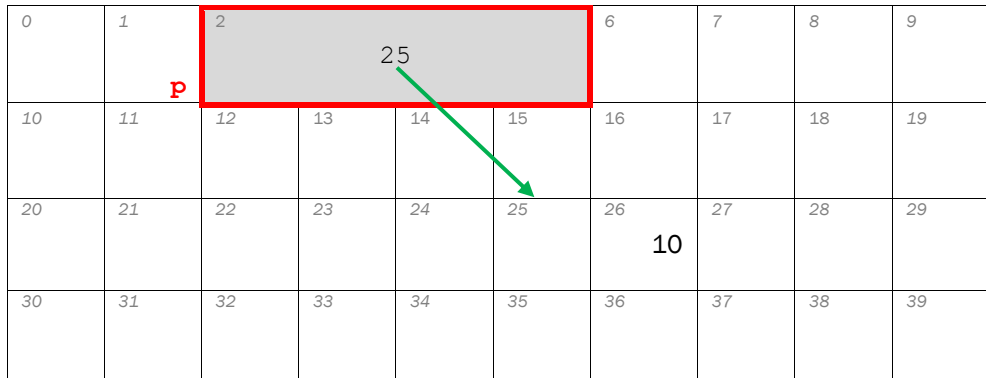


Figura 5.12. Mapa de memoria, libera memoria obtenida dinámicamente

Observemos que luego de ejecutar `delete`, la variable `p` aún mantiene su valor, la dirección 25, pero dicho espacio de memoria ya no nos pertenece. Incluso, entre las direcciones 25 y 28 persiste el valor 10, al cual ya no tenemos acceso.

Lo anterior podría compararse con la venta de una casa: luego de venderla continuamos teniendo su dirección, aunque ya no podemos volver a ingresar. El valor 10, que quedó en la memoria, sería como una pila de periódicos viejos que dejamos tirados en un rincón, es decir, basura que el nuevo dueño deberá eliminar.

5.2.1.11. Crear arrays dinámicamente

Los `array` son estructuras estáticas. Esto significa que luego de establecer su capacidad física, ésta no podrá ser modificada. Sin embargo, sí podemos crear un `array` de n elementos, donde n es una variable. Para esto debemos gestionar dinámicamente la memoria que utilizará el `array`, tratándolo como un puntero que apunta al primero de sus elementos, que de hecho, lo es.

En el siguiente código el usuario ingresa un valor entero n , el cual usamos para crear dinámicamente un `array` de n elementos tipo `int`.

```

int n;
cin >> n;

// creo un array de n enteros
int* a = new int[n];

// inicializo los n elementos del array
for(int i=0; i<n; i++)
{
    a[i]=0;
}

```

5.2.1.12. Redimensionar un array

Como el array es una estructura estática y su capacidad no puede ser modificada, no es posible redimensionar un array. Pero si lo tratamos dinámicamente podemos crear un array *a* con una determinada capacidad física, y crear otro array *b* con mayor capacidad que el anterior. Luego, copiar los elementos del array *a* al array *b*, liberar *a* y hacer que *a* apunte a *b*. Así, tendremos la ilusión de haber redimensionado (o agrandado) el array *a*.

```

int n;
cin >> n;

int* a = new int[n];

// duplico la capacidad del array
redimensionar(a,n,2*n);

```

Veamos la función `redimensionar`:

```

void redimensionar(int*& a,int cap,int nuevaCap)
{
    // creo un array con la nueva capacidad
    int* b = new int[nuevaCap];

    // copio los datos del array a al array b
    for(int i=0; i<cap; i++)
    {

```



```

        b[i]=a[i];
    }

    // libero el array a
    delete a;

    // hago que a apunte al nuevo array
    a = b;
}

```

5.2.1.13. Crear matrices dinámicamente

Previamente, al estudiar los *arrays* multidimensionales, hemos mencionado que un *array* de dos dimensiones (una matriz) es un *array* de *arrays*. Como un *array* es en sí mismo la referencia al primero de sus elementos, una matriz es un puntero a punteros. Así, para crear dinámicamente una matriz tendremos que crear el *array* principal, y luego crear uno a uno los *arrays* que se desprenden a partir de cada uno de sus elementos.

```

// el usuario ingresa una cantidad de filas y columnas
int filas,columnas;
cin >> filas >> columnas;

// creo el array principal (filas)
int** mat = new int*[filas];

// recorro las filas y los arrays secundarios (columnas)
for(int i=0;i<filas; i++)
{
    mat[i] = new int[columnas];
}

```

Finalmente, accedemos a cualquiera de las celdas de la matriz `mat` indicando fila y columna. Por ejemplo, si queremos asignar el valor 10 a la celda 2,1 haremos:

```
mat[2][1]=10;
```

5.2.2. Actividad práctica: TAD Array



El TAD `Array`, cuya implementación quedará a cargo del lector, representa un `array` tipo `T` cuya capacidad física que se incrementará automáticamente cada vez que su longitud (cantidad de elementos que contiene) se aproxime a dicha capacidad, proporcionándole al usuario la ilusión de estar trabajando con una estructura de datos dinámica.



TAD Array

Un ejemplo de uso de este TAD sería el siguiente:

```
Array<int> a = array<int>();
arrayAdd<int>(a,1);
arrayAdd<int>(a,2);
arrayAdd<int>(a,3);

for(int i=0; i<arraySize<int>(a); i++)
{
    int* x = arrayGet<int>(a,i);
    cout << *x << endl;
}
```

Observemos que `arrayGet` retorna la dirección del *i*-ésimo elemento del `array`. Esto nos permitirá acceder a dicho elemento y, eventualmente, modificar su valor. Como en el siguiente ejemplo donde asignamos 0 (cero) a todos los elementos del `array` cuyo valor es mayor a 10.

```
for(int i=0; i<arraySize<int>(a); i++)
{
    int* x = arrayGet<int>(a,i);
    if( *x>10 )
    {
        *x = 0;
    }
}
```

5.2.3. Actividad práctica: TAD Map



Un *map* consiste en una colección de pares {*key*, *value*}. Cada elemento (*value*) que agreguemos al *map* quedará acompañado de una clave (*key*) a través de la cual posteriormente podremos recuperarlo, modificarlo o eliminarlo.

El desarrollo del TAD, cuya especificación descargaremos mediante el código QR de la derecha, quedará a cargo del lector. Aquí veremos un ejemplo de uso de esta estructura de datos que será, por lejos, una de más flexibles, prácticas y funcionales.



TAD Map

Básicamente *Map* provee dos funciones: *mapPut* y *mapGet*, que respectivamente permiten agregar una nueva entrada y acceder una entrada ya existente.

En el siguiente ejemplo usamos un *map* para vincular valores enteros con cadenas de caracteres. Como ya mencionamos, cada elemento que agreguemos a la estructura deberá estar acompañado de una clave que lo identificará.

```
// creo un map y le agrego valores
Map<string,int> m = map<string,int>();

mapPut<string,int>(m,"uno",1);
mapPut<string,int>(m,"dos",2);
mapPut<string,int>(m,"tres",3);
```

En este ejemplo creamos un *map* que vincula números (*value*) con cadenas de caracteres (*key*). Por eso, al declararlo indicamos: `Map<string,int> m`.

La clave (*key*) siempre deberá ser de un tipo primitivo, *string* o cualquier otro que soporte el uso de los operadores relacionales `<`, `>`, `==` y `!=`.

Ahora le solicitaremos al usuario que ingrese una cadena y la usaremos para determinar si el *map* contiene o no una entrada vinculada a dicha *key*.

```
// el usuario ingresa una clave
string k;
cin >> k;
```

```
// verifico si existe y muestro su valor asociado
if( mapContains<string,int>(m,k) )
{
    int* v = mapGet<string,int>(m,k);
    cout<< "El valor asociado a " << k <<" es " << *v <<endl;
```

Si existía una entrada vinculada con la clave que el usuario ingresó, le permitiremos modificar el valor asociado a dicha clave.

```
// el usuario ingresa un valor para modificar el anterior
int nuevoVal;
cin >> nuevoVal;

// modificamos el valor asociado a k
*v = nuevoVal;
}
```

El *map* debe preservar el orden en el que hemos agregado sus entradas. Así, al iterarlo accederemos a cada una de ellas en el mismo orden en que fueron creadas.

```
// itero el map mostrando sus valores
mapReset<string,int>(m);
while( mapHasNext<string,int>(m) )
{
    string k = mapNextKey<string,int>(m);
    int* v = mapGet<string,int>(m,v);
    cout << k <<" , " << *v << endl;
}
```

5.2.4. Autoevaluación y ejercicios



Autoevaluación



Ejercicios

5.3. Lección 14

En esta lección, estudiaremos estructuras de datos dinámicas, que pueden crecer o decrecer a medida que agreguemos o quitemos datos. Según sea la disposición que este tipo de estructuras utilice para guardar el conjunto de datos, podremos clasificarlas en *lineales* o *no lineales*. Pero, básicamente, cualquiera sea su clasificación, todas surgen de la concatenación de unidades de información llamadas *nodo*. En este capítulo nos concentraremos en estudiar *estructuras dinámicas lineales*.

5.3.1. Nodo

Llamamos *nodo* a una estructura (`struct`) que, además de contener datos, incluye la dirección de memoria de uno o varios nodos de su mismo tipo.

No importa el nombre de la estructura. Puede llamarse `Nodo`, `NodoPersona`, o simplemente `X`. Lo que la convierte en un nodo es el hecho de que tenga entre sus campos la dirección de memoria de otro/s nodo/s de su mismo tipo de dato.



Lección 14

Veamos la estructura `Nodo` que tiene dos campos: `info` y `sig`. El primero permite guardar un valor tipo `int`; el segundo, la dirección de otro `Nodo`.

```
struct Nodo
{
    int info;           // dato que guarda el nodo
    Nodo* sig;         // puntero a otro Nodo
};
```

Recordemos que el nombre de la estructura en sí mismo no determina que sea un nodo. Lo que la convierte en nodo es el hecho de tener una referencia a otro nodo de su mismo tipo. En este caso, dicha referencia es el campo `sig`.

Como un nodo tiene un dato y la dirección de otro nodo, que a su vez tiene un dato y la dirección de otro nodo, y así sucesivamente, podemos plantear una estructura llamada *lista de nodos enlazados* o simplemente *lista enlazada*.

Al ser `info` tipo `int` se restringe el tipo de dato de los elementos que la estructura podrá contener. Por cuestiones didácticas aceptaremos esta limitación, pero más adelante desarrollaremos una API basada en *templates* genéricos.

5.3.2. Lista enlazada (linked list)

Siendo `p` un `Nodo*` que contiene la dirección de memoria de un nodo, podemos imaginar una lista enlazada que se desprende a partir del nodo direccionado por `p`. Observemos que el último nodo tiene en su campo `sig` el valor `NULL` (representado con un carácter X), reflejando que se trata del último elemento de la lista.



Figura 5.13. Lista enlazada

En primer lugar, es muy importante comprender que la lista en sí misma no existe. Sólo existe la dirección de un nodo que tiene la dirección del siguiente, que direcciona al siguiente, y así sucesivamente. En segundo lugar, también hay que entender que el objetivo de esta estructura es mantener en memoria una colección de elementos (en este caso) tipo `int`.

Quiero decir, la lista no es una lista de nodos, sino una lista de enteros, cada uno de los cuales está contenido dentro de un nodo. El nodo es un contenedor.

Cómo se crea la lista lo veremos a continuación. Ahora, asumiendo que la lista ya está creada y tiene varios elementos, analizaremos un algoritmo para recorrerla y mostrar el valor que contiene cada uno de sus nodos.

5.3.2.1. Recorrer una lista enlazada

La lógica consiste en desplazar un puntero auxiliar a través de cada uno de los nodos de la lista, comenzando desde el primero. Para esto declaramos `aux` y le asignamos `p`, donde `p` es el puntero que direcciona el primer nodo de la lista (`p` es la lista).

```
Nodo* aux = p; // aux apunta al primer nodo de la lista
```

Luego de esta declaración, tendremos dos punteros que direccionan al primer nodo de la lista enlazada.

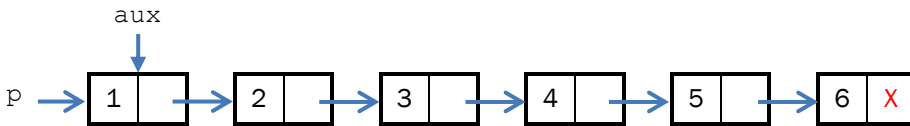


Figura 5.14. Puntero auxiliar apuntando al primer nodo de la lista

El siguiente paso será mostrar el valor que contiene el nodo direccionado por `aux`, y hacer que `aux` pase a apuntar al siguiente nodo. La salida por pantalla será 1.

```
// muestro el valor del nodo
cout << aux->info << endl;

// hago que aux apunte al siguiente nodo de la lista
aux = aux->sig;
```

La sentencia `aux=aux->sig` desplaza el puntero `aux` hacia el siguiente nodo de la lista, que ahora se verá así (con `aux` apuntando al segundo nodo):

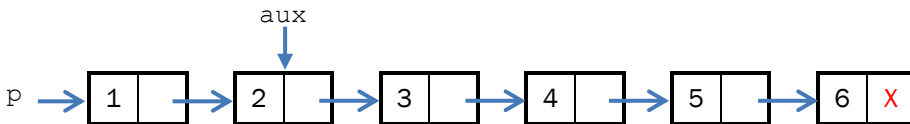


Figura 5.15. Puntero auxiliar apuntando al segundo nodo de la lista

Si repetimos el código anterior, la salida por pantalla será 2, y el puntero `aux` quedará direccionando al tercer nodo de la lista:

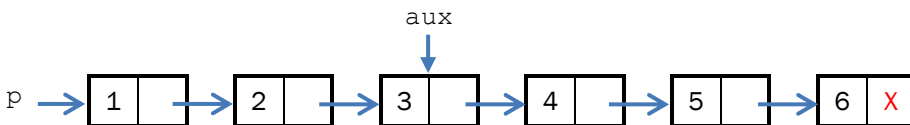


Figura 5.16. Puntero auxiliar apuntando al tercer nodo de la lista

Al cabo de varias iteraciones, `aux` quedará apuntando al último nodo de la lista, como apreciamos en la siguiente figura:

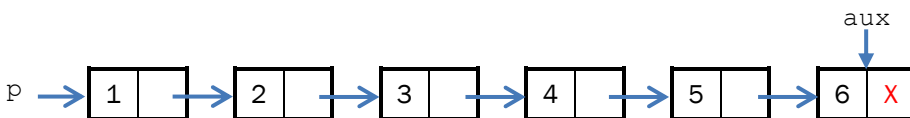


Figura 5.17. Puntero auxiliar apuntando al último nodo de la lista

En este estado, si repetimos el proceso anterior la salida por pantalla será 6 y `aux` quedará en `NULL`, reflejando que ya no quedan más nodos por recorrer.

El código de la función `mostrar`, que recorre la lista enlazada y muestra por consola el valor de cada uno de sus nodos, es el siguiente:

```
void mostrar(Nodo* p)
{
    Nodo* aux = p;
    while( aux!=NULL )
    {
        cout << aux->info << endl;
        aux = aux->sig;
    }
}
```

5.3.2.2. Agregar un valor al final de una lista enlazada

Siendo `p` un puntero tipo `Nodo*` que apunta al primer nodo de una lista ya creada y con elementos, analizaremos un algoritmo que nos permitirá agregar un nuevo valor `v` al final de la lista direccionada por `p`.

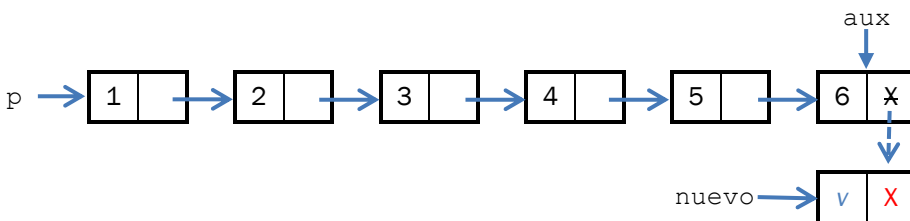


Figura 5.18. Se agrega un nuevo nodo al final de la lista

La imagen refleja el siguiente algoritmo:

1. Creamos un nuevo nodo (`nuevo`) y le asignamos `v` a `info` y `NULL` a `sig`.
2. Con un puntero auxiliar (`aux`) recorremos la lista hasta llegar al último nodo, que identificamos porque su campo `sig` es `NULL`.
3. Asignamos `aux->sig=nuevo`, logrando así que el nodo direccionado por `nuevo` pase a formar parte de la lista, ocupando el último lugar.

En la siguiente tabla, a la izquierda, vemos el código de la función `agregar`. A la derecha, un programa principal que crea una lista enlazada, le agrega elemen-

tos, muestra en pantalla su contenido, y libera la memoria gestionada dinámicamente. Esta última función (`liberar`) la analizaremos más adelante.

```
Nodo* agregar(Nodo*& p,int v)
{
    // creo un nuevo nodo
    Nodo* nuevo = new Nodo();
    nuevo->info = v;
    nuevo->sig = NULL;

    // la lista ya existe?
    if( p!=NULL )
    {
        // busco el ultimo
        Nodo* aux = p;
        while(aux->sig!=NULL)
        {
            aux = aux->sig;
        }

        // enlace el nuevo nodo
        aux->sig = nuevo;
    }
    else
    {
        // nuevo es el primero
        p = nuevo;
    }

    return nuevo;
}
```

```
int main()
{
    // creo una lista vacia
    Nodo* p = NULL;

    // le agrego elementos
    agregar(p,1);
    agregar(p,2);
    agregar(p,3);
    agregar(p,4);
    agregar(p,5);
    agregar(p,6);

    // muestro por pantalla
    mostrar(p);

    // liberar la memoria
    liberar(p);

    return 0;
}
```

En la función `agregar` usamos un `if` para diferenciar dos casos:

- Si la *lista existe* (`p` es distinto de `NULL`).
- Si la *lista no existe* (`p` es igual a `NULL`).

Dentro del primer caso, implementamos el algoritmo que analizamos previamente. En el segundo caso, como la lista no existe porque `p` es `NULL`, el nuevo nodo se convertirá en el primer y único nodo de la lista, y debe ser direccionado por `p`. Por esta razón, la función `agregar` recibe al puntero `p` por referencia.

5.3.2.3. Liberar la memoria que ocupa la lista

Hemos visto que una lista enlazada no es más que un puntero direccionando al primero de sus nodos. Si la lista está vacía, el valor del puntero será `NULL`. Cada elemento que agregamos a la lista requiere crear dinámicamente un nuevo nodo y enlazarlo. Esta memoria debe ser reintegrada al sistema operativo cuando ya no la necesitamos, sea que eliminemos un único elemento o la lista completa.

Para esto, desarrollaremos la función `liberar`, cuyo código veremos a continuación, que implementa el siguiente algoritmo para liberar la memoria que ocupan todos los nodos de la lista.

1. Mientras que `p` sea distinto de `NULL`:
 - a. Para liberar el nodo direccionado por `p`, primero debemos guardar la dirección del siguiente, haciendo: `Nodo* aux=p->sig`.
 - b. Liberamos el nodo direccionado por `p`: `delete p`;
 - c. Ahora que el primer nodo ya no forma parte de la lista, haremos que el segundo nodo, direccionado por `aux`, sea el primero: `p=aux`.

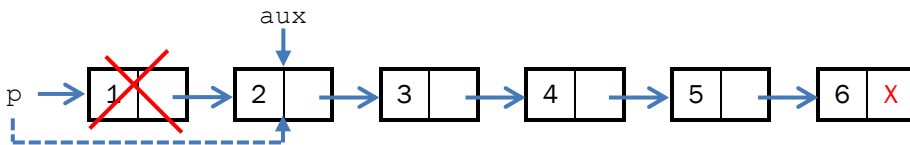


Figura 5.19. Se libera el primer nodo de la lista

Cuando `p` apunte al último (y único) nodo de la lista, su campo `sig` será `NULL` por lo que al asignar `p=aux`, `p` pasará a ser `NULL` y el `while` finalizará.

```
void liberar(Nodo*& p)
{
    while( p!=NULL )
    {
        Nodo* aux = p->sig;
        delete p;
        p = aux;
    }
}
```

Luego de invocar a `liberar`, la memoria que utilizaba la lista enlazada ya no formará parte del programa, y `p` (la lista) quedará en `NULL`.

5.3.2.4. Determinar si la lista contiene un valor especificado

La función `buscar` recorre la lista y retorna la dirección del nodo que contiene la primera ocurrencia del valor `v`, o `NULL` si ninguno de sus nodos contiene dicho valor.

```
Nodo* buscar(Nodo* p, int v)
{
    Nodo* aux = p;
    while( aux!=NULL && aux->info!=v )
    {
        aux = aux->sig;
    }

    return aux;
}
```

5.3.2.5. Insertar un valor en una lista ordenada

La función `insertarOrdenado` recorre la lista (que debe estar ordenada o vacía) buscando el primer elemento que resulte ser mayor al valor que queremos insertar. Entonces, lo inserta, actualizando las referencias (punteros) que corresponda.

La implementación de esta función respeta el siguiente algoritmo:

1. Creamos un nuevo nodo y le asignamos el valor a insertar.
2. Recorremos la lista con un puntero auxiliar hasta encontrar un valor mayor al que vamos a insertar, asegurándonos de guardar el puntero al nodo anterior.
3. Al finalizar la recorrida, si el puntero al nodo anterior es `NULL`, será que la lista estaba vacía o el valor a insertar es menor que el primero.
4. En cambio, si al finalizar la recorrida el puntero auxiliar quedó en `NULL` significa que estamos intentando insertar un valor mayor que el último.
5. Si no se verifican los puntos 3 ni 4, el valor que estamos por insertar quedará colocado en el medio de la lista, es decir, no será ninguno de los extremos.

Analizaremos gráficamente cómo queda la lista y qué debemos hacer en las situaciones que se corresponden con los puntos 3, 4 y 5. En todos los casos, aceptaremos que la lista tiene los siguientes elementos:



Figura 5.20. Lista enlazada

Situación 3: La lista está vacía o el elemento a insertar es menor que el primero.

Supongamos que queremos insertar el valor 1, que claramente debe quedar ubicado como primer elemento de la lista. En esta situación, el puntero `ant` será `NULL` (no aparece en la imagen), y los punteros `p` y `aux` apuntarán al primer nodo. El nuevo nodo (que contiene a 1) estará direccionado por `nuevo`.

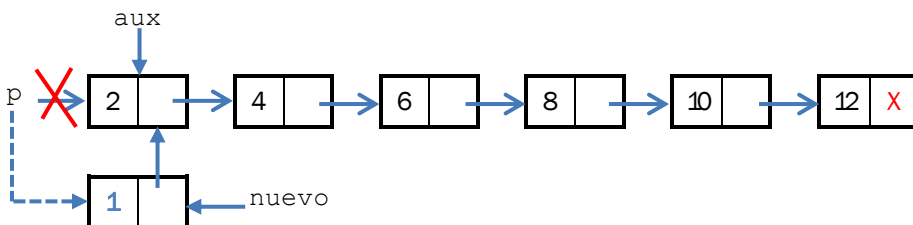


Figura 5.21. Se inserta un nodo al inicio de la lista

Para que `nuevo`, que contiene a 1, quede ubicado como primer nodo de la lista, debemos hacer que `p` apunte a `nuevo` y el campo `sig` de `nuevo` apunte al “ex” primer nodo, que aún permanece direccionado por `aux`.

Situación 4: Debemos insertar un valor que es mayor que todos los elementos de la lista; por lo tanto, corresponde ubicarlo al final. Supondremos que este valor será 14.

En esta situación, `aux` será `NULL` (no aparece en la imagen) y `ant` quedará apuntando al último nodo de la lista. El nuevo nodo está direccionado por `nuevo`.

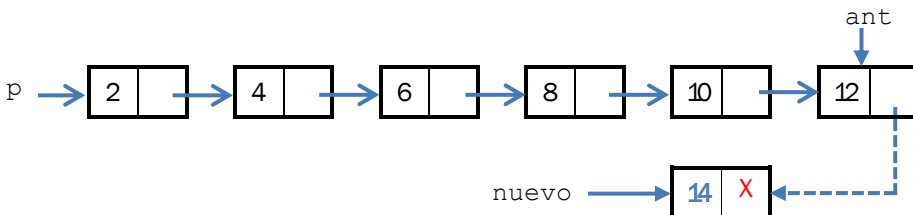


Figura 5.22. Se inserta un nodo al final de la lista

Para colocar el nuevo nodo al final de la lista haremos: `ant->sig=nuevo`. Sólo resta colocar `NULL` en el campo `sig` del nuevo nodo: `nuevo->sig=NULL`.

Situación 5: Aquí el valor a insertar (supondremos que es 7) es intermedio; por lo que el nodo que lo contenga se ubicará por la mitad de la lista. Al llegar a esta situación, `aux` apuntará al nodo cuyo valor será inmediatamente mayor que 7 y `ant` apuntará al nodo anterior. El nodo direccionado por `nuevo` debe quedar ubicado entre `ant` y `aux`. Para esto haremos: `nuevo->sig=aux` y `ant->sig=nuevo`.

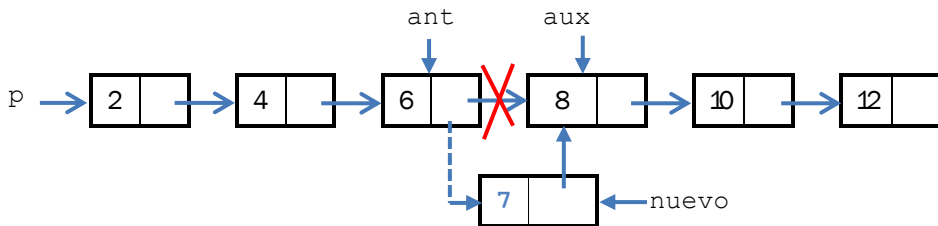


Figura 5.23. Se inserta un nodo en el medio de la lista

```
Nodo* insertar(Nodo*& p,int v)
{
    Nodo* nuevo = new Nodo();
    nuevo->info = v;

    Nodo* ant=NULL;
    Nodo* aux=p;
    while( aux!=NULL&& aux->info<v)
    {
        ant = aux;
        aux = aux->sig;
    }

    if( ant==NULL )
    {
        nuevo->sig = p;
        p = nuevo;
    }
    else
    {
        ant->sig = nuevo;
        if( aux==NULL )
        {
            nuevo->sig = NULL;
        }
        else
        {

```

```

        nuevo->sig=aux;
    }
}

return nuevo;
}

```

5.3.2.6. Eliminar un elemento de la lista

Debemos recorrer la lista con un puntero auxiliar, procurando guardar el puntero al nodo anterior. Así, cuando `aux` apunte al nodo que vamos a eliminar y `ant` apunte al nodo inmediatamente anterior, la operación consistirá en hacer que el campo `sig` de `ant` pase a apuntar al nodo direccionado por el campo `sig` de `aux`. Luego, podremos eliminar (con `delete`) el nodo direccionado por `aux`.

En la siguiente figura, aceptando que queremos eliminar de la lista el valor 6, `ant` quedará apuntando al nodo que contiene a 4 y `aux` al nodo que vamos a eliminar.

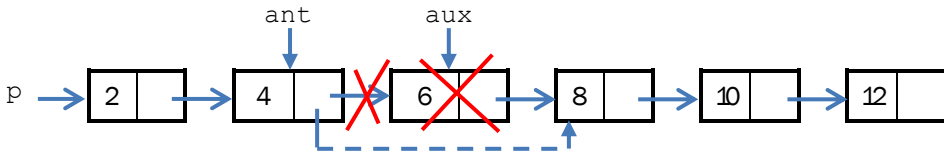


Figura 5.24. Se elimina un nodo de la lista

La función `eliminar` recorre la lista buscando la primera ocurrencia de un determinado valor y remueve el nodo que lo contiene.

```

void eliminar(Nodo*& p,int v)
{
    Nodo* aux=p;
    Nodo* ant=NULL;

    while( aux!=NULL && aux->info!=v )
    {
        ant = aux;
        aux = aux->sig;
    }

    if( ant==NULL )
    {

```

```

        p = aux->sig;
    }
    else
    {
        ant->sig = aux->sig;
    }

    delete aux;
}

```

Observemos que fue necesario colocar un `if` para determinar el caso particular en que el elemento a remover sea el primero. En tal situación, `ant` será `NULL` y `aux` apuntará al primer nodo de la lista.

5.3.3. Actividad práctica: API de operaciones sobre listas enlazadas



Las siguientes funciones, cuya especificación podremos descargar a través del código QR de la derecha, son la generalización de las operaciones que hemos estudiado. Aceptaremos que formarán parte de nuestra biblioteca de funciones y, como ya es habitual, su implementación quedará a cargo del estudiante.



API de operaciones
sobre listas enlazadas

Todas las funciones de la API trabajarán sobre la siguiente estructura de nodo:

```

template<typename T>
struct Node
{
    T info;
    Node<T>* sig;
};

```

A modo de ejemplo, analizaremos cómo desarrollar una de estas funciones.

La función `add`, que agrega un elemento al final de una lista enlazada tiene el siguiente prototipo:

```
Node<T>* add(Node<T>* p, T t);
```

La función agrega el valor `t` al final de la lista direccionada por `p`, retornando la dirección de memoria del nodo que contiene el valor que se agregó.

Un ejemplo de uso sería el siguiente:

```
Node<int>* p = NULL;
add<int>(p, 1);
add<int>(p, 2);
add<int>(p, 3); // p->{1,2,3}
```

La implementación de la función será:

```
template<typename T>
Node<T>* add(Node<T>*& p, T t)
{
    // creo un nuevo nodo
    Node<T>* nuevo = new Node<T>();
    nuevo->info = t;
    nuevo->sig = NULL;

    // la lista ya existe?
    if( p!=NULL )
    {
        // busco el ultimo
        Node<T>* aux = p;
        while(aux->sig!=NULL)
        {
            aux = aux->sig;
        }

        // enlace el nuevo nodo
        aux->sig = nuevo;
    }
    else
    {
        // nuevo es el primero
        p = nuevo;
    }

    return nuevo;
}
```


5.3.4. Pila (stack)

Una pila (*stack*) es una estructura de datos restrictiva, también denominada LIFO (*Last In First Out*). Esta estructura restringe el modo en que se agrega o remueve cada elemento del conjunto de datos. Justamente, LIFO indica que el último elemento que ingresó en la pila también será el primero en salir.

Para imaginar esta estructura, pensemos en cómo formaríamos una pila de libros. Tomamos el libro A y lo colocamos sobre la mesa. Luego, tomamos el libro B y lo colocamos sobre A. Posteriormente, colocamos el libro C sobre B, que ya estaba colocado sobre A. Por último, apilamos el libro D, que quedará ubicado sobre C.

Si queremos tomar un libro de la pila, naturalmente, tomaremos el que quedó en la cima, es decir, el D. Si tomamos otro libro sería el turno de C; luego, de B; y finalmente, de A. De este modo, verificamos que el último libro que entró en la pila fue el primero en salir (LIFO); el anteúltimo que ingresó fue el segundo en salir, el antepenúltimo fue el tercero, y así sucesivamente.

Podemos implementar la estructura pila, con sus dos operaciones asociadas: *push* (apilar) y *pop* (desapilar) sobre una lista enlazada.

5.3.4.1. Apilar un elemento (*push*)

A través de esta operación agregaremos un nuevo elemento a la estructura. El algoritmo consiste en colocar el nuevo elemento en la primera posición del conjunto, desplazando hacia el final a todos los otros elementos ya existentes.

Lo anterior permite asegurar que el primer lugar de la estructura siempre estará ocupado por el último elemento que ingresó al conjunto; y cuando llegue el momento de remover un valor, éste será el que deberá salir (LIFO).

1. Creamos un nuevo nodo (*nuevo*), le asignamos el valor que queremos apilar y hacemos que su campo *sig* apunte al primer nodo: `nuevo->sig=p`.
2. Hacemos que *p* apunte al nuevo nodo: `p=nuevo`.

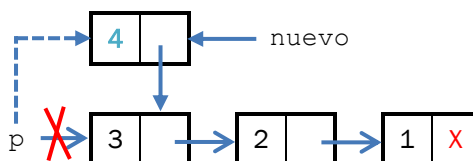


Figura 5.25. Se apila un nodo insertándolo al inicio de la lista

El código de la función apilar es el siguiente:

```
void apilar(Nodo*& p, int v)
{
    Nodo* nuevo = new Nodo();
    nuevo->info=v;
    nuevo->sig=p;
    p=nuevo;
}
```

5.3.4.2. Desapilar un elemento (pop)

Por las restricciones que definen a esta estructura no podemos escoger qué elemento queremos remover. Desapilar un elemento implica sí o sí remover el último que ingresó, el cual estará ubicado en el primer lugar de la lista enlazada.

1. Guardamos la dirección de *p* en un puntero auxiliar: *aux=p*.
2. Hacemos que *p* apunte al segundo nodo de la lista: *p=p->sig*.
3. Guardamos el valor del (ex) primer nodo y lo liberamos: *delete aux*.

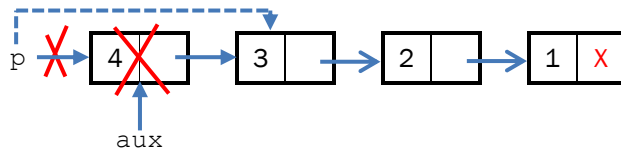


Figura 5.26. Se desapila un nodo removiéndolo del inicio de la lista

La implementación de esta operación la vemos a continuación.

```
int desapilar(Nodo*& p)
{
    Nodo* aux = p;
    p = p->sig;
    int v = aux->info;
    delete aux;
    return v;
}
```

5.3.4.3. Determinar si la pila tiene elementos

Podemos considerar una tercera operación que nos permita determinar si la pila está vacía o no. Su implementación es tan sencilla que se limita a evaluar si el puntero que direcciona la lista sobre la cual implementamos la pila es `NULL`.

```
bool esVacia(Nodo* p)
{
    return p==NULL;
}
```

5.3.4.4. Ejemplo de uso

En el siguiente código usamos una pila para invertir el orden de un conjunto de valores numéricos. La salida será: 3, 2, 1.

```
Nodo* p = NULL;
apilar(p,1);
apilar(p,2);
apilar(p,3);

while( !esVacia(p) )
{
    int v = desapilar(p);
    cout << v << endl;
}
```

5.3.5. Actividad práctica: API de operaciones sobre pilas (extensión)



Extenderemos la biblioteca de funciones de tratamiento de listas enlazadas con dos nuevas funciones: `push` y `pop`, las cuales, más adelante y dentro de este mismo capítulo, nos permitirán implementar el TAD `Stack`. La implementación de `push` y `pop`, tal como hemos venido trabajando, quedará a cargo del estudiante.



Extensión de la API de funciones sobre listas

5.3.6. Cola (queue)

Una cola (*queue*) es una estructura de datos restrictiva, también denominada FIFO (*First In First Out*). La estructura restringe el modo en que los datos deben ser ingre-

sados en el conjunto y, posteriormente, removidos. Justamente, FIFO indica que el primer elemento que ingresó debe ser también el primero en salir.

Para imaginar cómo funciona esta estructura de datos pensemos en la cola que se forma en la caja de un supermercado. Si no hay gente, la primera persona que llegue será atendida de inmediato. Pero si la atención demora demasiado tiempo, la próxima persona en llegar deberá formar una fila. Si la demora continuara y llegaran más personas se colocarían al final de la fila. Mientras tanto, la cajera atenderá al primero de la fila; luego, al segundo; y así sucesivamente. La fila es una *cola*.

Sólo existen dos operaciones asociadas a una cola: *enqueue* (encolar) y *dequeue* (desencolar). Adicionalmente, podemos plantear una tercera operación para determinar si la cola tiene elementos o no.

En este apartado, estudiaremos cómo implementar una cola utilizando una lista enlazada y dos punteros, que direccionan al primero y último nodo de la lista. Más adelante, veremos otra implementación que utiliza una *lista enlazada circular*.

5.3.6.1. Encolar un elemento (*enqueue*)

Esta implementación de cola trabaja sobre una lista enlazada con dos punteros que llamaremos *p* y *q*, direccionando respectivamente al primero y último nodo de la lista. La operación *encolar* se limita a agregar un nuevo nodo al final de la sublista direccionada por *q*, y posteriormente hacer que *q* apunte al nuevo nodo. De este modo *q* seguirá apuntando al último nodo de la lista.

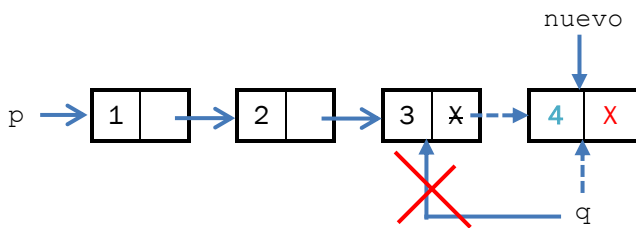


Figura 5.27. Se encola un nodo agregándolo al final de la lista

Antes de analizar el algoritmo, aclaremos que el puntero adicional que direcciona al último nodo de la lista busca maximizar su eficiencia. Pues, el hecho de disponer de un acceso directo a dicho nodo nos ahorra la necesidad de recorrer toda la lista cada vez que encolemos (agreguemos) un nuevo elemento.

El algoritmo consiste en los siguientes pasos:

1. Crear un nuevo nodo (nuevo).
2. Hacer que el último nodo de la lista apunte al nuevo: $q \rightarrow \text{sig} = \text{nuevo}$.
3. Hacer que q pase a apuntar al (nuevo) último nodo de la lista: $q = \text{nuevo}$.

```
void encolar(Nodo*& p, Nodo*& q, int v)
{
    Nodo* nuevo = new Nodo();
    nuevo->info = v;
    nuevo->sig = NULL;

    if( q!=NULL )
    {
        q->sig = nuevo;
    }
    else
    {
        p = nuevo;
    }

    q = nuevo;
}
```

Observemos que fue necesario agregar un `if` para determinar el caso particular en que la cola se encuentre vacía ($q == \text{NULL}$). En tal caso, al encolar un elemento, éste será el primero y el último; por lo cual, p y q deben apuntarlo.

5.3.6.2. Desencolar un elemento (*dequeue*)

La operación consiste en remover de la lista el primer nodo, que está direccionado por p ; y luego, hacer que p apunte al segundo nodo. El algoritmo es el siguiente:

1. Guardar en un puntero auxiliar la dirección del primer nodo: $\text{Nodo}^* \text{aux} = p$.
2. Hacer que p direcciona al segundo nodo: $p = p \rightarrow \text{sig}$.
3. Resguardar el valor del (ex) primer nodo y liberarlo: `delete aux`.

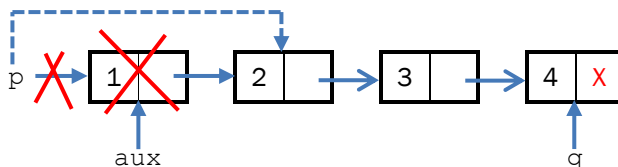


Figura 5.28. Se desencola un nodo removiéndolo del inicio de la lista

```

int desencolar(Nodo*& p,Nodo*& q)
{
    int ret = p->info;
    Nodo* aux = p;
    p = p->sig;
    delete aux;

    if( p==NULL )
    {
        q = NULL;
    }

    return ret;
}

```

En este caso, fue necesario agregar un `if` para determinar si el elemento que acabamos de desencolar era el único elemento que quedaba en la cola. Si así fuera, la habremos dejado vacía; por lo cual, `q` debe quedar en `NULL`.

5.3.6.3. Determinar si la cola tiene elementos

Esta operación es muy simple y su implementación se limita a determinar si `p` o `q` (cualquiera de los dos) es `NULL`.

```

bool esVacia(Nodo* p,Nodo* q)
{
    return p==NULL;
}

```

Observemos que para programar `esVacia` sólo utilizamos `p`. Aun así, por cuestiones de coherencia y prolijidad, la función debe recibir `p` y `q`; pues ambos punteros son la base sobre la cual está implementada de la estructura de datos.

Veamos un ejemplo:

```

// cola
Nodo* p = NULL;
Nodo* q = NULL;

```

```

encolar(p,q,1);
encolar(p,q,2);
encolar(p,q,3);

// SALIDA: 1, 2
cout << desencolar(p,q) << endl;
cout << desencolar(p,q) << endl;

encolar(p,q,4);
encolar(p,q,5);
encolar(p,q,6);

// SALIDA: 3, 4, 5, 6
while( !esVacia(p,q) )
{
    int i = desencolar(p,q);
    cout << i << endl;
}

```

5.3.6.4. Implementación sobre una lista circular

Una lista circular es una lista enlazada que tiene la particularidad de que el último elemento apunta al primero.

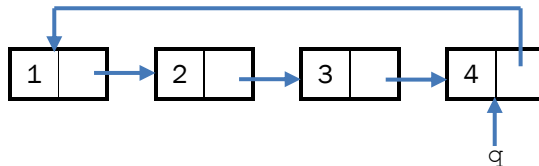


Figura 5.29. Lista circular

Si en vez de direccionar al primer nodo, direccionamos al último (es decir: siempre apuntamos al nodo que contiene el último elemento que se agregó a la lista), podremos tener direccionados ambos extremos: el último apuntado por *q* y el primero apuntado por el campo *sig* de *q*.

Encolar un elemento en una cola implementada sobre una lista circular

El algoritmo es el siguiente:

1. Creamos el nuevo nodo y le asignamos el valor a encolar.

2. El campo `sig` del nuevo nodo debe apuntar al primer elemento de la lista, que podemos identificar como `q->sig`. Es decir: `nuevo->sig=q->sig`.
3. Para colocar a `nuevo` como último nodo de la lista hacemos: `q->sig=nuevo`.
4. Finalmente, `q` debe apuntar al (nuevo) último nodo de la lista: `q=nuevo`.

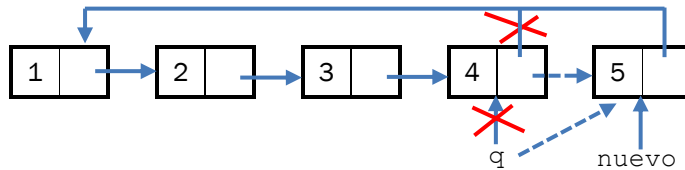


Figura 5.30. Se encola un nodo agregándolo al final de una lista circular

```
void encolar(Nodo*& q, int v)
{
    Nodo* nuevo = new Nodo();
    nuevo->info = v;

    if( q!=NULL )
    {
        nuevo->sig = q->sig;
        q->sig = nuevo;
    }
    else
    {
        nuevo->sig=nuevo;
    }

    q = nuevo;
}
```

Observemos que fue necesario agregar un `if` que nos permita determinar si la cola está vacía. De ser así, el elemento que estamos encolando será el primero y también el último; por lo cual, el campo `sig` de `nuevo` debe apuntar a sí mismo.

Desencolar un elemento en una cola implementada sobre una lista circular

El algoritmo es el siguiente:

1. Guardamos en un puntero auxiliar la dirección del primer nodo de la lista, que está referenciado por el campo `sig` de `q`: `Nodo* aux=q->sig`.

2. Hacemos que el `sig` de `q` pase a apuntar al segundo nodo, pues este será el nuevo primer nodo: `q->sig=aux->sig`.
3. Liberamos el (ex) primer nodo de la lista: `delete aux`.

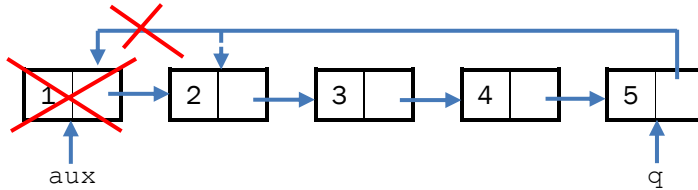


Figura 5.31. Se desencola un nodo removiéndolo del inicio de una lista circular

```
int desencolar(Nodo*& q)
{
    Nodo* aux = q->sig;
    q->sig = aux->sig;
    int ret = aux->info;

    if( aux==q)
    {
        q = NULL;
    }

    delete aux;
    return ret;
}
```

En este caso, fue necesario agregar un `if` para determinar si el nodo que desencolamos es el último elemento que quedaba en la cola. De ser así, `q` debe pasar a ser `NULL` para reflejar que la cola quedó vacía.

5.3.7. Actividad práctica: API de operaciones sobre colas (extensión)



Extenderemos la biblioteca de funciones de tratamiento de listas enlazadas, para agregar dos nuevas funciones: `enqueue` y `dequeue`, que más adelante y dentro de este mismo capítulo, nos permitirán implementar el TAD `Queue`. La implementación de estas funciones, tal como hemos venido trabajando, quedará a cargo del estudiante.



Extensión de la API de funciones sobre listas

5.3.8. Actividad práctica: TAD List, Stack y Queue



Desarrollaremos los TAD `List`, `Stack` y `Queue`, que nos permitirán encapsular las estructuras lista, pila y cola. La implementación de cada uno de estos TAD, cuyas especificaciones podremos descargar mediante el código QR de la derecha, quedará a cargo del lector.



TAD List, Stack y Queue

Simplemente, aquí analizaremos un pequeño ejemplo de uso de cada TAD aceptando que los mismos están programados y disponibles para ser utilizados.

Ejemplo de uso del TAD `List`:

```
List<int> lst = list<int>();
listAdd<int>(lst,1);
listAdd<int>(lst,2);
listAdd<int>(lst,3);

while( listHasNext<int>(lst) )
{
    int* e = listNext<int>(lst);
    cout << *e << endl;
}

listFree<int>(lst);
```

Ejemplo de uso del TAD `Stack`:

```
Stack<int> st = stack<int>();
stackPush<int>(st,1);
stackPush<int>(st,2);
stackPush<int>(st,3);

while( !stackIsEmpty<int>(st) )
{
    int e = stackPop<int>(st);
    cout << e << endl; // Salida: 3,2,1
}
```

Ejemplo de uso del TAD Queue:

```
Queue<int> q = queue<int>();
queueEnqueue<int>(q, 1);
queueEnqueue<int>(q, 2);
queueEnqueue<int>(q, 3);

cout << queueDequeue<int>(q) << endl; // Salida: 1
cout << queueDequeue<int>(q) << endl; // Salida: 2

queueEnqueue<int>(q, 4);
queueEnqueue<int>(q, 5);

while( !queueIsEmpty<int>(q) )
{
    int e = queueDequeue<int>(q);
    cout << e << endl; // Salida: 3,4,5
}
```

5.3.9. Autoevaluación y ejercicios



Autoevaluación



Ejercicios

5.4. Lección 15

El único objetivo de esta lección es mostrar algunos ejemplos que nos ayudarán a construir las estructuras de datos combinadas estudiadas en el capítulo 2, pero utilizando las colecciones que desarrollamos durante el presente capítulo.

5.4.1. Estructura de datos (parte 2)

Trabajaremos sobre la estructura `Persona` cuya declaración y función de inicialización veremos a continuación.



Lección 15

```
struct Persona
{
    int dni;
    string nombre;
};
```

```
Persona persona(int d,string n){ return {d,n}; }
```

5.4.1.1. Colección de estructuras

Cualquiera de los TAD que desarrollamos a lo largo de este capítulo nos permitirá mantener una colección de estructuras en memoria. Veamos algunos ejemplos.

Usando el TAD `Array`.

```
Array<Persona> aPers = array<Persona>();
arrayAdd<Persona>(aPers,persona(1010,"Pedro"));
arrayAdd<Persona>(aPers,persona(1020,"Pablo"));
arrayAdd<Persona>(aPers,persona(1030,"Juan"));
```

Usando el TAD `Map`, donde la *key* es el DNI y el *value* la persona.

```
Map<int,Persona> mPers = map<int,Persona>();
mapPut<int,Persona>(mPers,1010,persona(1010,"Pedro"));
mapPut<int,Persona>(mPers,1020,persona(1020,"Pablo"));
mapPut<int,Persona>(mPers,1030,persona(1030,"Juan"));
```

Usando el TAD `List`.

```
List<Persona> lPers = list<Persona>();
```

```
listAdd<Persona>(lPers,persona(1010,"Pedro"));
listAdd<Persona>(lPers,persona(1020,"Pablo"));
listAdd<Persona>(lPers,persona(1030,"Juan"));
```

5.4.1.2. Colección de colecciones

Aquí, las combinaciones podrían ser tantas como imaginemos: un *map* de listas, un *map* de *array*, una lista de *map*, un *map* de *map*, etcétera.

A modo de ejemplo, analizaremos qué surge de la siguiente declaración:

```
Map<string,List<Persona> > mlPers;
```

La variable `mlPers` representa un *map* que vincula *keys* tipo `string` con listas de personas. Cada entrada estará identificada por una clave que será un nombre, y el *value* será una lista de personas que se llaman de ese modo.

Esta estructura nos permitirá acceder, con cierta facilidad, a la lista de las personas que se llaman “Juan”, o a la lista de quienes se llaman “Pedro”, etcétera.

```
// creamos el map
Map<string,List<Persona> > mlPers =
    map<string,List<Persona> >();

// agregamos una entrada para "Juan" con una lista vacia
List<Persona>* lst = mapPut<string,List<Persona> >(
    mlPers
    ,"Juan"
    ,list<Persona>());

// agrego personas que se llaman Juan a la lista
listAdd<Persona>(*lst,persona(11,"Juan Molina"));
listAdd<Persona>(*lst,persona(22,"Juan Rodriguez"));
listAdd<Persona>(*lst,persona(33,"Juan Cardozo"));

// agregamos una entrada para "Pedro" con una lista vacia
lst = mapPut<string,List<Persona> >(
    mlPers
    ,"Pedro"
    ,list<Persona>());
```

```
// agrego personas que se llaman Pedro a la lista
listAdd<Persona>(*lst,persona(44,"Pedro Ramirez"));
listAdd<Persona>(*lst,persona(55,"Pedro Picapiedra"));
listAdd<Persona>(*lst,persona(66,"Pedro Nolasco"));

// mostramos las personas que se "llaman Juan"
lst = mapGet<string,List<Persona> >(mPers,"Juan");

listReset<Persona>(*lst);
while( listHasNext<Persona>(*lst) )
{
    Persona* p = listNext<Persona>(*lst);
    cout << p->nombre << endl;
}
```

5.4.1.3. Colección de estructuras que tienen colecciones

Para estudiar este caso modificaremos la estructura `Persona` del siguiente modo:

```
struct Persona
{
    int dni;
    string nom;
    List<string> hijos; // agregamos una lista de cadenas
};

Persona persona(int d,string n,List<string> l)
{
    return {d,n,l};
}
```

Analicemos qué estructura genera la siguiente declaración:

```
Map<int,Persona> mPers = map<int,Persona>();
```

Se trata de un *map* de estructuras `Persona`, cada una de las cuales tiene una lista de cadenas de caracteres.

En el siguiente ejemplo usamos el *map* `mPers` para vincular personas (*value*) con números de DNI (*key*). El simple hecho de conocer el DNI de una persona nos dará acceso a todos sus datos, incluyendo la lista con los nombres de sus hijos.

```

// creamos el map
Map<int, Persona> mPers = map<int, Persona>();

// agregamos una persona
Persona* p = mapPut<int, Persona>(
    mPers
    ,1010
    ,persona(1010,"Carlos",list<string>()));

// agrego los hijos de la persona
listAdd<string>(p->hijos,"Sergio");
listAdd<string>(p->hijos,"Marcos");
listAdd<string>(p->hijos,"Adrian");

// agregamos otra persona con sus hijos
p = mapPut<int, Persona>(
    mPers
    ,2020
    ,persona(2020,"Rodolfo",list<string>()));

listAdd<string>(p->hijos,"Esteban");
listAdd<string>(p->hijos,"Karina");

// mostramos las personas y sus hijos
mapReset<int, Persona>(mPers);
while( mapHasNext<int, Persona>(mPers) )
{
    int dni = mapNextKey<int, Persona>(mPers);
    Persona* x = mapGet<int, Persona>(mPers,dni);
    cout << dni << " " << x->nom << endl;

    // hijos
    List<string> hijos = x->hijos;
    listReset<string>(hijos);
    while( listHasNext<string>(hijos) )
    {
        string* h = listNext<string>(hijos);
        cout << *h << endl;
    }
}

```

5.4.2. Autoevaluación y ejercicios



Autoevaluación



Ejercicios

5.5. ¿Qué sigue?

Paradójicamente, contrariando al Teorema de la Programación Estructurada, este capítulo no tiene una única salida. Pues, como ya mencionamos, debemos retomar el capítulo 4 para resolver, una vez más, todos los problemas que allí se plantearon; pero aplicando, en este caso, los conocimientos y herramientas que adquirimos aquí.

Luego sí, será el turno del capítulo 6, donde estudiaremos un algoritmo complejo diseñado para comprimir información; tal como lo hace el programa WinRAR. Comprender la lógica del algoritmo de Huffman (ese es su nombre) e implementarlo será el corolario de este Curso de Algoritmos y Programación A Fondo.